

**A PROJECT REPORT
ON
“DEEPFAKE IMAGE DETECTION”**

**Submitted to
KIIT Deemed to be University
In Partial Fulfillment of the Requirement for the
Award of
BACHELOR’S DEGREE IN
INFORMATION TECHNOLOGY
BY**

ADITYA ANAND	2106005
MD SAUD KHAN	2106039
PRIYANSHU SRIVASTAVA	2106046
SUDIP MONDAL	2106073
SURAJIT HATI	2106074
SHASHANK GUPTA	2106303

**UNDER THE GUIDANCE OF
DR. PRATYUSA MUKHERJEE
Assistant Professor (II)**



**SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA – 751024
April 2025**

KIIT Deemed to be University
School of Computer Engineering
Bhubaneswar, Odisha - 751024



CERTIFICATE

This is to certify that the project entitled
“DEEPFAKE IMAGE DETECTION”

Submitted by

ADITYA ANAND	2106005
MD SAUD KHAN	2106039
PRIYANSHU SRIVASTAVA	2106046
SUDIP MONDAL	2106073
SURAJIT HATI	2106074
SHASHANK GUPTA	2106303

is a record of Bonafide work carried out by them, in the partial fulfillment of the requirement for the award of Degree of Bachelor of Engineering (Information Technology) at KIIT Deemed to be university, Bhubaneswar. This work is done during year 2024- 2025, under the guidance.

Date: 03/04/2025

Dr. Pratyusa Mukherjee
Assistant Professor (II)

Acknowledgements

We are profoundly grateful to **Dr. Pratyusa Mukherjee** of KIIT Deemed to be University for her expert guidance and continuous encouragement throughout to see that this project rights its target since its commencement to its completion.

ADITYA ANAND

MD SAUD KHAN

PRIYANSHU SRIVASTAVA

SUDIP MONDAL

SURAJIT HATI

SHASHANK GUPTA

ABSTRACT

Deepfake technology, powered by artificial intelligence, has raised significant concerns due to its potential for misuse in misinformation, identity fraud, and media manipulation. This project aims to develop a deep learning-based deepfake image detection system to distinguish between real and AI-generated images. The model is built using DenseNet121, a pre-trained convolutional neural network (CNN), and trained on a dataset sourced from Kaggle, consisting of 6,000 images split into training (4,200), validation (900), and testing (900) sets.

To enhance model robustness, data augmentation techniques such as rotation, flipping, and normalization were applied. The model was optimized using categorical cross-entropy loss and the Adam optimizer, with early stopping and regularization mechanisms to prevent overfitting. Performance was evaluated using metrics like accuracy, loss, precision, recall, and F1-score, achieving an accuracy of 93% on test data.

For real-world usability, the trained model was deployed as a Streamlit-based web application, enabling users to upload images and receive classification results in real time. The system was tested extensively before deployment to ensure reliability. Challenges such as dataset imbalance, model overfitting, and deployment constraints were addressed through techniques like class balancing, hyperparameter tuning, and model optimization.

Future enhancements of this project include real-time deepfake detection in videos, improved accuracy with transformer-based architectures, and integration with cybersecurity systems. This project contributes to the growing need for deepfake detection tools, providing a scalable and efficient solution for combating AI-generated media manipulation.

Keyword : Deepfake, DenseNet121, cybersecurity, Detection, convolutional neural network (CNN)

CONTENTS

1	Introduction	7-8
1.1	Overview of Deepfake Technology	7
1.2	Need for Deepfake Detection	8
2	Basic Concepts/ Literature Review	9-11
2.1	Key Goals	9-10
2.1.1	Develop a Robust Deep Learning Model	9
2.1.2	Leverage Sensor Pattern Noise (SPN) for Enhanced Detection	9
2.1.3	Optimize Dataset Processing and Augmentation	9
2.1.4	Deploy a User-Friendly Web Interface	9-10
2.1.5	Evaluate Model Performance and Interpretability	10
2.1.6	Contribute to Digital Forensics and Ethical AI	10
2.2	Expected Outcomes	10-11
2.2.1	Accurate and Efficient Deepfake Detection Model	10
2.2.2	Practical and Scalable Web-Based Application	10
2.2.3	Improved Detection Using SPN Analysis	10
2.2.4	Comprehensive Performance Evaluation	10-11
2.2.5	Contribution to the AI and Cybersecurity Community	11
3	Problem Statement / Requirement Specifications	12-15
3.1	Project Planning	12
	Project Analysis	12
	Design Constraints	13
	System Architecture OR Block Diagram	13
4	Implementation	16-27
4.1	Methodology OR Proposal	16-17
4.1.1	Model Architecture Selection	16
4.1.2	Model Implementation	16
4.1.3	Model Training	17
4.1.4	Model Evaluation	17
4.2	Dataset Preparation	17-19
4.2.1	Data Collection	18
4.2.2	Data Preprocessing	18
4.2.3	Data Splitting and Augmentation	18
4.2.4	Data Augmentation	18-19
4.3	Model Testing before deployment	19-22
4.3.1	Loading the Trained Model	19
4.3.2	Preprocessing Input Images	19

4.3.3	Making Predictions	19
4.3.4	Visualization of Results	19
4.3.5	The model was tested locally using the following Python script before being integrated into Streamlit	19
4.4	Results	22-24
4.4.1	Model Performance Metrics (Accuracy, Loss, Precision, Recall, and F1-Score)	22
4.4.2	Test Results and Observations	23
4.4.3	Performance Graphs (Accuracy vs. Epochs, Loss vs. Epochs)	23-24
4.5	Deployment Using Streamlit	24-27
4.5.1	Web Application Interface	24-25
4.5.2	Prediction Workflow	25-26
4.5.3	User Interaction and Output Interpretation	26-27
4.6	Quality Assurance	27
5	Standards Adopted	28-29
5.1	Design Standards	28
5.2	Coding Standards	28
5.3	Testing Standards	29
6	Conclusion and Future Scope	30
6.1	Conclusion	30
6.2	Future Scope	30

List of Figures

- 1) Flowchart of Deepfake or Real Image Detection.
- 2) DeepFake detection website workflow.
- 3) Sequence diagram of Deepfake image detection process.
- 4) Entity-Relationship diagram for Deepfake detection system database schema.
- 5) Dataset split with equal distribution of real and deepfake images.
- 6) Testing code Snippet.
- 7) Model prediction result indicating the image as Deepfake.
- 8) Model prediction result indicating the image as Real.
- 9) Accuracy vs. Epochs Curve for Deepfake Image Detection Model.
- 10) Loss vs. Epochs Curve for Deepfake Image Detection Model.
- 11) Interface of DeepFake Image Detection.
- 12) Deepfake detection web app interface with uploaded image.
- 13) Deepfake Detection Web App Showing Prediction Result for the Uploaded Image.

Chapter 1

Introduction

This report details the development of a deep learning-based system for detecting deepfake images. The project uniquely leverages Sensor Pattern Noise (SPN), an intrinsic artifact resulting from camera sensor imperfections, to identify subtle cues in digital images that distinguish authentic content from manipulated ones. By integrating robust data preprocessing, state-of-the-art model architectures, and a user-friendly deployment framework, this work aims to enhance the capabilities of digital forensic tools and contribute to media authenticity verification.[1]

The motivation behind this project is to address the rising concerns over manipulated visual media. As deepfake technology becomes more accessible and its outputs increasingly realistic, traditional detection methods struggle to keep pace with these sophisticated alterations. This project proposes a novel approach by harnessing SPN as a digital fingerprint, thereby providing a reliable and explainable means of verifying image authenticity.[2]

1.1 Overview of Deepfake Technology

Deepfake technology refers to the application of advanced artificial intelligence and machine learning techniques—particularly generative adversarial networks (GANs) and autoencoders—to produce hyper-realistic manipulated images and videos. These techniques enable the synthesis of visual content where elements such as facial expressions, identities, and even entire scenes can be altered convincingly. Key characteristics of deepfakes include:

- **Realism:** As models are trained on large datasets, the output quality has significantly improved, making it difficult for the human eye to detect tampering.[2]
- **Accessibility:** With the democratization of AI tools and open-source software, deepfake creation has become accessible to a broader audience, lowering the barrier to entry for creating high-quality manipulated media.
- **Versatility:** Deepfakes have applications that range from harmless entertainment and artistic expression to malicious uses such as misinformation campaigns, fraud, and identity theft.

These advances, while offering innovative creative avenues, have also raised serious ethical, legal, and security concerns. As deepfakes become more pervasive, they challenge the integrity of visual information, creating a pressing need for robust detection and authentication mechanisms.

1.2 Need for Deepfake Detection

The emergence of deepfake technology has sparked significant concerns due to its potential misuse in spreading misinformation and manipulating public opinion. Deepfake detection is essential for several reasons:

- **Preserving Trust:** The ability to discern real content from fabricated media is critical to maintaining public trust in digital communications. With the rapid proliferation of deepfakes on social media and news platforms, effective detection tools help mitigate the risk of widespread misinformation.[3]

- **Digital Forensics:** In forensic investigations, verifying the authenticity of visual evidence is paramount. By exploiting SPN—unique to each camera—detection systems can reliably trace the source of an image, providing valuable insights during criminal investigations or disputes.
- **Mitigating Harm:** Beyond legal implications, deepfakes have the potential to inflict personal and reputational harm. An effective detection mechanism protects individuals and organizations from the adverse consequences of false or manipulated imagery.[3]
- **Regulatory and Ethical Considerations:** As governments and organizations work toward establishing norms for digital content authenticity, advanced detection systems can serve as a critical component in regulatory frameworks, ensuring that deepfake technology is used responsibly.

In this context, the project not only focuses on achieving high detection accuracy through the DenseNet121-based model but also emphasizes the explainability and practical utility of the approach[3]. By combining SPN analysis with deep learning, the detection system provides an innovative solution that is both robust and adaptable to the evolving challenges posed by deepfake technology.

Chapter 2

Basic Concepts/ Literature Review

The primary objective of this project is to develop an efficient and accurate deepfake image detection system using deep learning techniques and Sensor Pattern Noise (SPN) analysis. The project aims to provide a reliable solution for distinguishing real images from manipulated ones while ensuring scalability and practical usability[4]. By leveraging advanced machine learning models, this detection system enhances digital forensic capabilities and contributes to the fight against misinformation and fraudulent media.

2.1 Key Goals

2.1.1 Develop a Robust Deep Learning Model

This project employs the DenseNet121 architecture to construct a deepfake detection model with high accuracy. By utilizing transfer learning, the model can leverage pre-trained features to improve its classification ability on deepfake datasets. To enhance generalization and prevent overfitting, techniques such as dropout and L2 regularization are applied[5]. The model is trained on a diverse dataset with real and deepfake images to ensure its adaptability to different types of manipulations.

2.1.2 Leverage Sensor Pattern Noise (SPN) for Enhanced Detection

Sensor Pattern Noise (SPN) serves as a unique fingerprint for authenticating images, as every camera sensor introduces subtle noise patterns. This project incorporates SPN analysis to detect inconsistencies in manipulated images[5], strengthening the model's ability to distinguish deepfakes. By analyzing noise variations and artifacts introduced by deepfake generation methods, the model achieves improved detection accuracy compared to traditional classification approaches.

2.1.3 Optimize Dataset Processing and Augmentation

Effective dataset management is crucial for training an accurate deepfake detection model. The project employs preprocessing techniques such as image normalization and resizing to standardize inputs. Data augmentation techniques[6], including rotation, zooming, and flipping, are implemented to increase dataset diversity and improve model robustness. The dataset is carefully split into training, validation, and testing sets to ensure balanced learning and reliable performance evaluation.

2.1.4 Deploy a User-Friendly Web Interface

To make deepfake detection accessible to users, a Streamlit-based web application is developed. The interface allows users to upload images and receive real-time classification results, providing a simple yet effective tool for detecting manipulated content. The model is seamlessly integrated into the web app, ensuring smooth processing and user-friendly

interactions[7]. The application also displays confidence scores to help users interpret the reliability of the predictions.

2.1.5 Evaluate Model Performance and Interpretability

A thorough evaluation of the model is conducted using key performance metrics such as accuracy, precision, recall, and F1-score. Training and validation curves are analyzed to assess the model's learning behaviour and potential overfitting issues[8]. The project also emphasizes transparency by providing confidence scores for detections, allowing users to understand the basis of classification decisions. The effectiveness of SPN-based analysis is also compared against traditional deep learning approaches to validate its contribution to detection accuracy.

2.1.6 Contribute to Digital Forensics and Ethical AI

This project aligns with the broader goal of promoting ethical AI and strengthening digital forensics. By developing a reliable deepfake detection system, the project aids forensic experts, media analysts, and cybersecurity professionals in identifying manipulated content. Additionally[9][10], the research insights gained can be used to enhance future deepfake detection technologies, contributing to the global fight against misinformation and deceptive media.

2.2 Expected Outcomes

2.2.1 Accurate and Efficient Deepfake Detection Model

The primary outcome of the project is a deep learning-based model capable of detecting deepfake images with a high degree of accuracy—ideally above 90%. Such performance indicates the model's robustness and its ability to distinguish between authentic and manipulated images in a reliable manner[12][13]. The model utilizes sophisticated feature extraction through pre-trained architectures (e.g., DenseNet121) and learns the subtle differences between real and fake images. Achieving high accuracy ensures its applicability in real-world environments where deepfakes may be more complex and harder to identify[15].

2.2.2 Practical and Scalable Web-Based Application

Another major outcome is the development of a user-friendly, web-based application using Streamlit[16][17]. This application allows users to upload images directly through a graphical interface and receive immediate feedback on whether the image is a deepfake or not. The web app is lightweight, responsive, and deployable across various platforms, making it practical for public use, as well as scalable for institutional integration[17]. The use of a browser-based framework also ensures accessibility without the need for installation or complex setup.

2.2.3 Improved Detection Using SPN Analysis

Sensor Pattern Noise (SPN) analysis enhances the detection mechanism by examining the intrinsic noise patterns generated by image sensors during image capture. These noise patterns are unique to each camera and difficult to replicate in synthetic images[18]. By

integrating SPN-based analysis, the system improves its ability to detect subtle inconsistencies in fake images, particularly those that are not easily caught through conventional feature-based analysis. This technique adds another layer of reliability and strengthens the model's capacity to identify manipulated content with greater precision.

2.2.4 Comprehensive Performance Evaluation

An essential part of the system development is evaluating its performance using various metrics and validation methods[19][20]. The outcome includes a thorough analysis of model accuracy, loss curves during training, confusion matrices, and performance comparisons against other existing models or baseline approaches. This ensures the model is not only accurate but also transparent and reliable in terms of how it performs across different datasets and scenarios. The evaluation helps stakeholders understand the model's strengths, limitations, and areas for future improvement.

2.2.5 Contribution to the AI and Cybersecurity Community

The final expected outcome is the contribution this project makes to the broader AI and cybersecurity communities. By presenting a functional and research-backed deepfake detection system, the project offers practical insights and tools for digital forensics[21], journalism, and online content moderation. It can assist in preventing the spread of misinformation and forged media by acting as a verification tool. Additionally, the research can serve as a foundation for future advancements in AI-driven media authentication, policy making, and public awareness[23].

Chapter 3

Problem Statement / Requirement Specifications

With the rapid advancement of artificial intelligence, the generation of synthetic media such as deepfake images has become increasingly realistic and accessible. This poses significant challenges in various domains, including security, media integrity, and privacy. The primary objective of this project is to develop an automated system capable of detecting deepfake images using deep learning techniques. The system should accept input images and accurately classify them as either real or deepfake, providing a confidence score along with the prediction. To achieve this, convolutional neural networks (CNNs) or other suitable deep learning models will be employed, trained on a well-labeled dataset containing both authentic and manipulated images. The implementation will use Python, with TensorFlow or PyTorch frameworks, and will include preprocessing steps such as resizing and normalization of images. The system will be evaluated based on standard metrics like accuracy, precision, recall, and F1-score to ensure robustness. Additionally, the design will prioritize scalability and adaptability to various datasets while maintaining efficient performance during both training and inference. A simple user interface, either GUI or command-line based, may also be developed to enhance usability.

3.1 Project Planning

The project will be executed in several phases to ensure systematic development and timely completion. Initially, extensive research and dataset collection will be conducted to gather a balanced set of real and deepfake images. This will be followed by data preprocessing, including resizing, normalization, and augmentation. The next phase involves designing and training a deep learning model, preferably using CNNs with TensorFlow or PyTorch. Once trained, the model will be tested and evaluated using metrics such as accuracy and F1-score. Finally, a simple interface may be developed to facilitate user interaction with the detection system. Each phase will include testing and review to ensure accuracy, efficiency, and scalability.

3.2 Project Analysis

The project addresses the growing threat posed by deepfake images, which are increasingly used for malicious purposes such as misinformation, identity theft, and defamation. Traditional detection methods are no longer sufficient due to the sophistication of modern generative AI techniques. Hence, a deep learning-based approach is essential for accurate and automated detection. By analyzing patterns and inconsistencies in image data using convolutional neural networks, the system can effectively differentiate between real and fake images. This analysis confirms the feasibility of the proposed solution and highlights the importance of leveraging advanced AI models to counter deepfake threats in real time.

3.3 Design Constraints

The design of the deepfake image detection system is subject to several constraints. Firstly, the availability and quality of labeled datasets significantly impact model performance, as a biased or limited dataset can reduce accuracy. Secondly, deep learning models, particularly CNNs, require substantial computational resources and time for training, which may limit development on low-end systems. Additionally, the model must balance detection accuracy with efficiency to ensure real-time or near-real-time predictions. Compatibility with various image formats and adaptability to new deepfake generation techniques are also important constraints. Lastly, ethical considerations must be taken into account to ensure the responsible use of the technology.

3.4 System Architecture OR Block Diagram

The flowchart outlines the Deepfake Image Detection workflow, starting with image upload and preprocessing. The image is then analyzed by a trained deep learning model to classify it as real or fake. The result is displayed to the user, ensuring a quick and efficient detection process.

Deepfake-or-Real Image Detection Flowchart

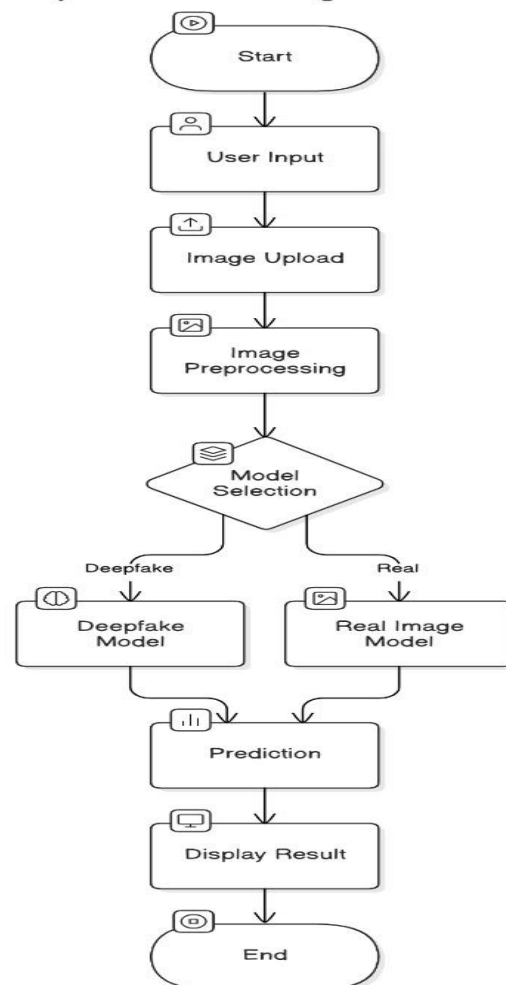


Figure 1: Flowchart of Deepfake or Real Image Detection[5]

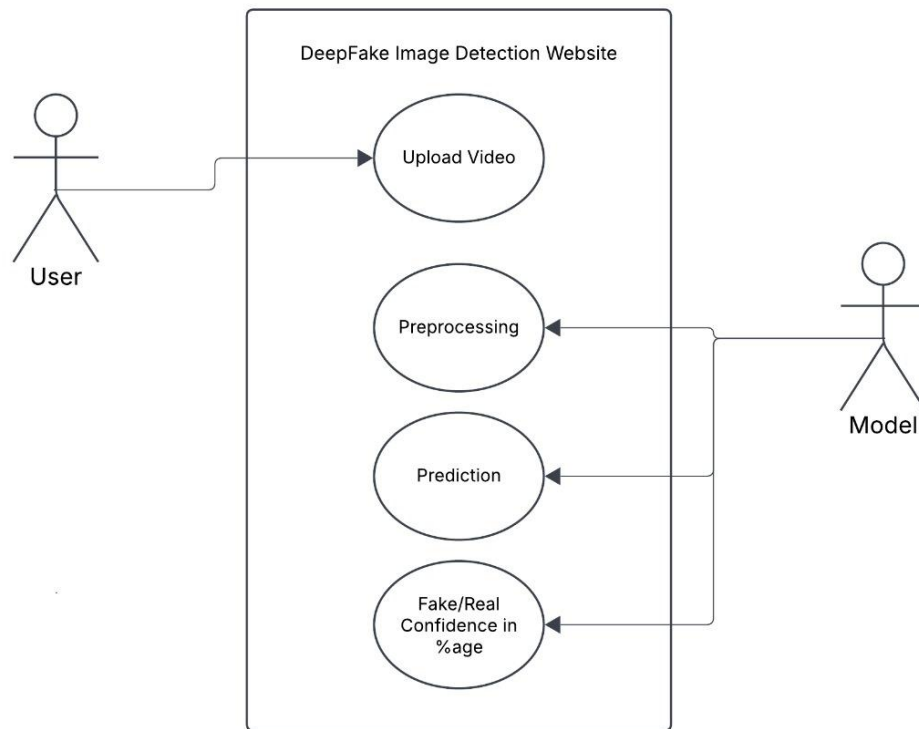


Figure 2: DeepFake detection website workflow[7]

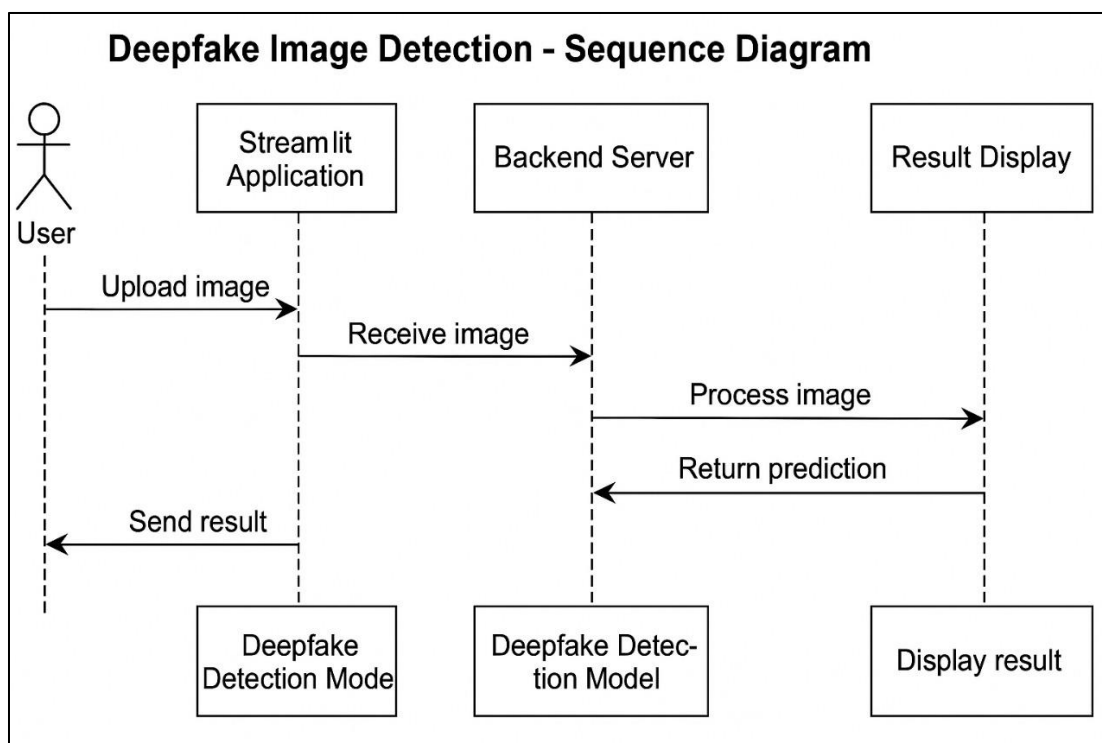


Figure 3: Sequence diagram of Deepfake image detection process[10]

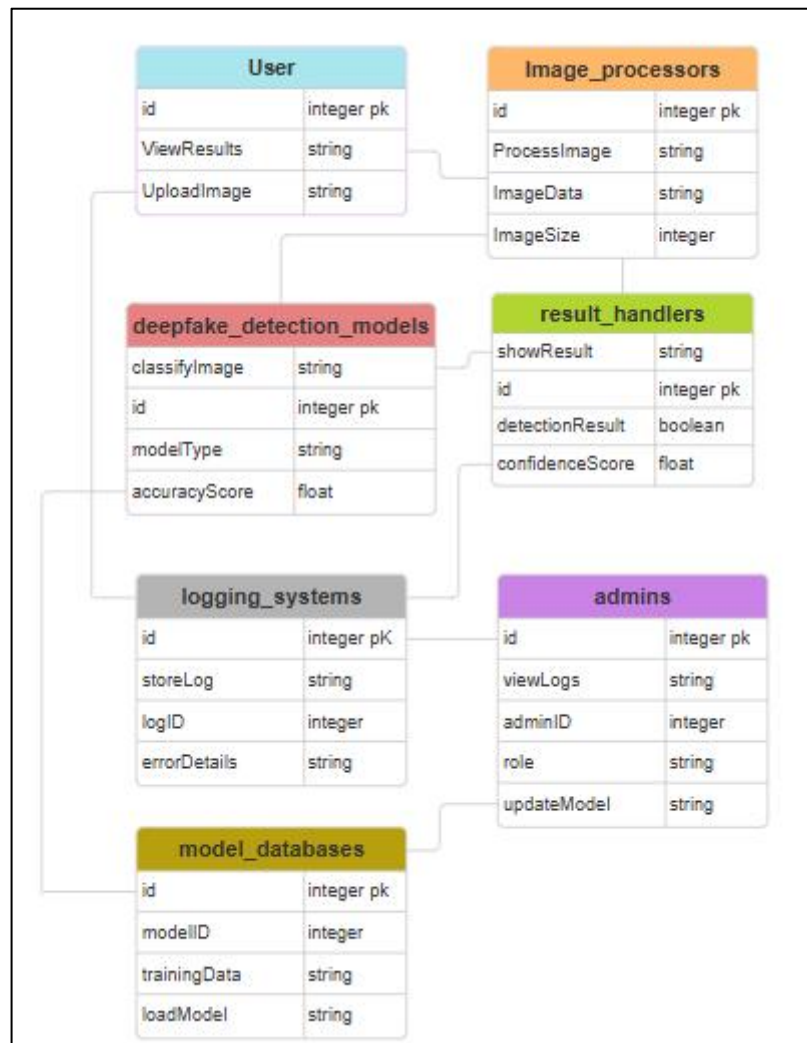


Figure 4: Entity-Relationship diagram for Deepfake detection system database schema[15]

Chapter 4

Implementation

The Deepfake detection model is developed using deep learning techniques, leveraging the DenseNet121 architecture for efficient feature extraction and classification. This process involves selecting an appropriate model architecture, implementing it with the necessary modifications, training it with optimized parameters, and evaluating its performance to ensure reliability in real-world applications.

4.1 Methodology OR Proposal

4.1.1 Model Architecture Selection

To build a robust deepfake detection system, the choice of model architecture is crucial. DenseNet121 (Densely Connected Convolutional Network) was selected as the backbone due to its unique ability to facilitate efficient feature propagation, reduce the number of parameters, and enhance gradient flow across layers. Unlike traditional convolutional neural networks (CNNs), where each layer only connects to the next, DenseNet connects each layer to every other layer, ensuring that information is passed more effectively across the network. This results in better feature reuse, reduced overfitting, and improved model efficiency. Additionally, DenseNet121 has been pre-trained on the ImageNet dataset, providing a strong foundation for transfer learning. This approach significantly speeds up training while also improving accuracy, as the model can leverage previously learned features relevant to image classification tasks.

The pre-trained DenseNet121 model is modified by removing its fully connected (FC) layers and adding a customized classification head tailored for binary classification. The extracted features are processed using a Global Average Pooling (GAP) layer, which helps reduce the spatial dimensions of the feature maps while retaining the most important information. This is followed by a fully connected dense layer with 128 neurons and ReLU activation, enhancing the model's ability to differentiate deepfake images from real ones. To mitigate overfitting, dropout layers with a 50% rate are introduced, ensuring that the model generalizes well on unseen data. The final classification is evenly performed using a single neuron output layer with a sigmoid activation function, producing a probability score that determines whether an image is real or manipulated.

4.1.2 Model Implementation

The model is implemented using TensorFlow and Keras, ensuring an efficient deep learning workflow. The DenseNet121 backbone is loaded with ImageNet weights, and the top layers are replaced with a new classification head. Initially, all layers of the DenseNet121 model are frozen to prevent modifications to the pre-trained weights. However, to fine-tune the model for deepfake detection, the last 15 layers of DenseNet121 are unfrozen, allowing them to be trained on the specific dataset. Fine-tuning is a crucial step as it helps the model adapt to the characteristics of deepfake images while still retaining the general visual features learned from ImageNet.

To compile the model, the Adam optimizer is used due to its adaptive learning rate capabilities, which help in achieving faster convergence. The binary cross-entropy loss function is selected as it is well-suited for binary classification problems. The model's performance is tracked using accuracy as the primary evaluation metric, ensuring that improvements are monitored throughout the training process.

4.1.3 Model Training

The training process involves feeding batches of images into the network, where each image undergoes a series of transformations as it passes through the DenseNet121 layers. The dataset consists of 6000 images, which are split into 4200 for training, 900 for validation, and 900 for testing. To improve generalization and prevent overfitting, data augmentation techniques such as random rotations, width and height shifts, shear transformations, zooming, and horizontal flips are applied. This ensures that the model learns to recognize deepfake images under various distortions, improving its robustness against unseen manipulations.

The model is trained for 21 epochs with a batch size of 64, balancing computational efficiency and learning stability. Early stopping is enabled with a patience of 5 epochs, ensuring that the training halts when the validation loss stops improving. This prevents unnecessary computation and reduces the risk of overfitting. Throughout the training, the model continuously updates its internal weights using backpropagation and gradient descent, progressively refining its ability to differentiate deepfake images from real ones.

4.1.4 Model Evaluation

Once training is complete, the model undergoes rigorous evaluation on the test dataset containing 900 images to assess its real-world performance. The model achieves an accuracy of 93%, indicating strong proficiency in distinguishing deepfake images. Additional metrics such as loss values and confusion matrices are analyzed to understand the model's behavior further. The confusion matrix provides insight into false positives and false negatives, helping identify cases where the model misclassifies deepfake images as real or vice versa.

To visualize the model's learning progress, training history plots are generated, showcasing accuracy and loss trends over epochs. The accuracy plot reveals how the model improves over time, while the loss plot indicates how well it minimizes classification errors. The results demonstrate that the model effectively converges with minimal overfitting, thanks to techniques like dropout layers, early stopping, and data augmentation.

With a well-structured model architecture, optimized training process, and strong evaluation results, this deepfake detection system provides a reliable solution for identifying manipulated images, making it a valuable tool in the fight against digital misinformation.

4.2 Dataset Preparation

A well-structured and diverse dataset is critical for training an effective deepfake detection model. This project utilizes a dataset sourced from Kaggle ([Deepfake and Real Images](#)), which contains a large collection of real and deepfake images. From this dataset, 6,000 images were selected to train and evaluate the model. The dataset was carefully pre-processed, split into training, validation, and testing sets, and augmented to improve model generalization. These steps ensure that the model can robustly identify manipulated images across various deepfake generation techniques.

4.2.1 Data Collection

The dataset contains real images from authentic sources and deepfake images generated using AI-based manipulation techniques. The 6,000 images selected for this project include an equal number of real and deepfake images to maintain class balance. This ensures that the model learns to detect deepfake features without being biased toward one class.

4.2.2 Data Preprocessing

To ensure uniformity and optimize computational efficiency, several preprocessing steps are applied:

- **Resizing:** All images are resized to 256×256 pixels to maintain consistency across the dataset.
- **Normalization:** Pixel values are scaled to the range [0,1] by dividing by 255, ensuring stable gradient updates during model training.
- **Noise and Artifacts Removal:** Basic filtering techniques are applied to remove unwanted noise while preserving subtle deepfake artifacts.
- **Format Standardization:** All images are converted to RGB format to ensure compatibility with the deep learning model.

4.2.3 Data Splitting and Augmentation

The dataset is split into three subsets for efficient training and evaluation:

Data Split	Number of Images	Real Images (%)	Deepfake Images (%)
Training	4,200	50%	50%
Validation	900	50%	50%
Test	900	50%	50%

Figure 5: Dataset split with equal distribution of real and deepfake images.

To maintain a balanced representation, the dataset is split using the `train_test_split` function, ensuring each subset contains an equal proportion of real and deepfake images.

4.2.4 Data Augmentation

To improve the model's robustness and prevent overfitting, various augmentation techniques are applied using `ImageDataGenerator` in TensorFlow. These include:

- **Rotation (up to 20°):** Helps the model recognize images at different orientations.
- **Width and Height Shifts (up to 20%):** Introduces variations in object positioning.
- **Shear Transformations:** Alters the shape of images to simulate different angles and perspectives.
- **Zooming (up to 20%):** Helps the model identify deepfake patterns at different scales.
- **Horizontal Flipping:** Enhances recognition of mirrored deepfake images.

These transformations simulate real-world variations, improving the model's ability to detect deepfakes under different conditions. With this structured dataset, preprocessing pipeline, and

augmentation strategy, the model is well-equipped to learn distinguishing features between real and deepfake images, ensuring accurate and reliable detection.

4.3 Model Testing before deployment

Before deploying the deepfake detection model using Streamlit, it was essential to test its performance on individual images. This ensured that the model was functioning correctly and providing accurate predictions. To achieve this, a Python script was used to load the trained model, preprocess images, and generate predictions.

4.3.1 Loading the Trained Model

- The trained model was loaded using TensorFlow's `load_model()` function from the saved .h5 file.
- This allowed for direct inference without retraining, ensuring consistent performance.

4.3.2 Preprocessing Input Images

- The test image was resized to 256x256 pixels, matching the input dimensions expected by the model.
- Pixel values were normalized by scaling between 0 and 1 to maintain consistency with the training process.
- A batch dimension was added since TensorFlow models require input in batch format.

4.3.3 Making Predictions

- The pre-processed image was passed to the model, which generated a probability score.
- If the probability was greater than 0.5, the image was classified as Real; otherwise, it was classified as Deepfake.

4.3.4 Visualization of Results

- The predicted label was printed to verify the classification.
- Matplotlib was used to display the test image along with its predicted category, providing a visual confirmation of the results.

By running multiple test cases with real and deepfake images, the model's accuracy was verified before proceeding with deployment. After confirming that the model provided reliable predictions, it was integrated into a Streamlit web application for user-friendly interaction.

4.3.5 The model was tested locally using the following Python script before being integrated into Streamlit:

```
•[14]: from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

model = load_model("deepfake_detector_model.h5")

IMG_HEIGHT = 256
IMG_WIDTH = 256

def predict_image(image_path, model):
    img = image.load_img(image_path, target_size=(IMG_HEIGHT, IMG_WIDTH))

    img_array = image.img_to_array(img) / 255.0

    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)

    if prediction[0] > 0.5:
        return "Real"
    else:
        return "Deepfake"

image_path = "/Users/shashangupta/Downloads/Dataset/Test/Fake/fake_1.jpg"
result = predict_image(image_path, model)

print(f"Prediction: {result}")

img = image.load_img(image_path)
plt.imshow(img)
plt.title(f"Prediction: {result}")
plt.show()
```

Figure 6: Testing code Snippet

- Running this script ensured that the model worked correctly before being deployed in a production environment.

Output are as following:

1/1 ————— 1s 705ms/step

Prediction: Deepfake

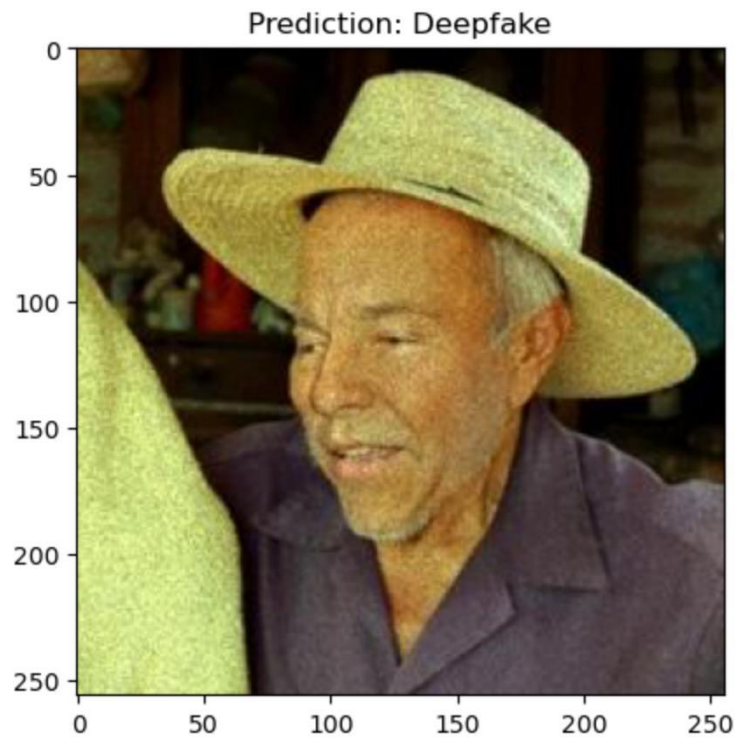


Figure 7: Model prediction result indicating the image as Deepfake

1/1 ————— 1s 793ms/step

Prediction: Real

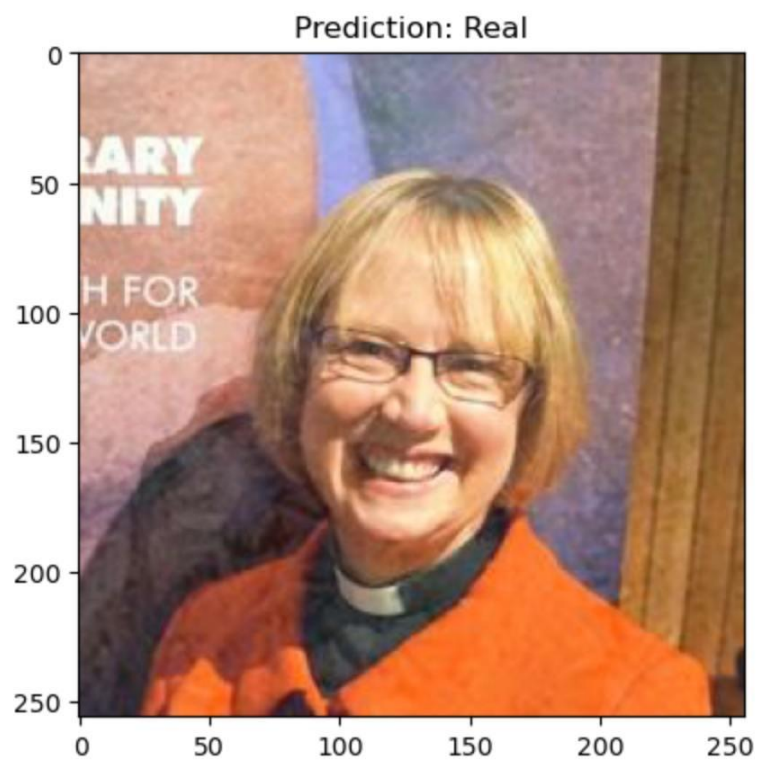


Figure 8: Model prediction result indicating the image as Real

By overcoming these challenges, the deepfake detection model was successfully optimized and deployed, making it accessible for real-world applications.

4.4 Results

Evaluating the performance of the deepfake detection model was essential to ensure its accuracy and reliability in real-world applications. The assessment was conducted using various performance metrics, test results, and graphical representations of accuracy and loss over different epochs. This section details the evaluation methodology, observations, and final conclusions drawn from the model's performance.

4.4.1 Model Performance Metrics (Accuracy, Loss, Precision, Recall, and F1-Score)

To measure the effectiveness of the trained model, several key performance metrics were considered:

Accuracy: The model achieved an accuracy of **93%** on the test dataset, indicating a high capability to distinguish between real and deepfake images. Accuracy is the proportion of correctly classified images out of the total test images.

Loss: The model's loss was measured using **binary cross-entropy**, which calculates the error between predicted probabilities and actual labels. A lower loss value indicated that the model was making more confident and correct predictions.

Precision and Recall:

- **Precision** measures how many of the images classified as fake were actually fake, ensuring fewer false positives.
- **Recall** measures how many actual deepfake images were correctly detected by the model, indicating its effectiveness in identifying manipulated images.
- A higher precision and recall balance ensures fewer incorrect classifications and a lower false negative rate.

F1-Score: A balance between precision and recall, the F1-score provided a single value to assess the model's ability to detect deepfake images accurately while minimizing misclassifications.

Confusion Matrix: This matrix was used to visualize the classification results by breaking down true positives, true negatives, false positives, and false negatives. A strong diagonal dominance (high true positives and true negatives) in the confusion matrix confirmed the model's high reliability.

These metrics collectively provided a comprehensive understanding of how well the model performed in detecting deepfake images and areas where potential improvements could be made.

4.4.2 Test Results and Observations

After training, the model was tested using a separate dataset of 900 images to evaluate its performance under unseen conditions. Several key observations were made:

- The model correctly classified the majority of deepfake and real images, demonstrating its robustness and ability to generalize well on new data.
- Some misclassifications occurred, primarily in cases where fake images were highly realistic, particularly those generated with advanced deepfake techniques that retained subtle facial details.
- Lighting conditions, image resolution, and compression artifacts affected classification accuracy, indicating that model performance could be further enhanced with additional preprocessing techniques.
- The model showed higher confidence in detecting real images compared to fake ones, suggesting that deepfake images with minimal alterations were more challenging to classify.
- Data Augmentation and Regularization helped minimize overfitting, allowing the model to maintain similar accuracy levels on both training and validation datasets.

Overall, the test results confirmed that the deepfake detection model was highly effective, with room for further improvement in handling challenging deepfake images.

4.4.3 Performance Graphs (Accuracy vs. Epochs, Loss vs. Epochs)

To analyze the model's training behaviour and generalization capability, two critical performance graphs were plotted:

- **Accuracy vs. Epochs:** The accuracy graph showed an **initial steep rise**, indicating that the model was quickly learning key features in early epochs. After a certain point, the accuracy **plateaued**, suggesting that the model had reached its learning potential. Validation accuracy closely followed training accuracy, confirming that the model did not overfit.

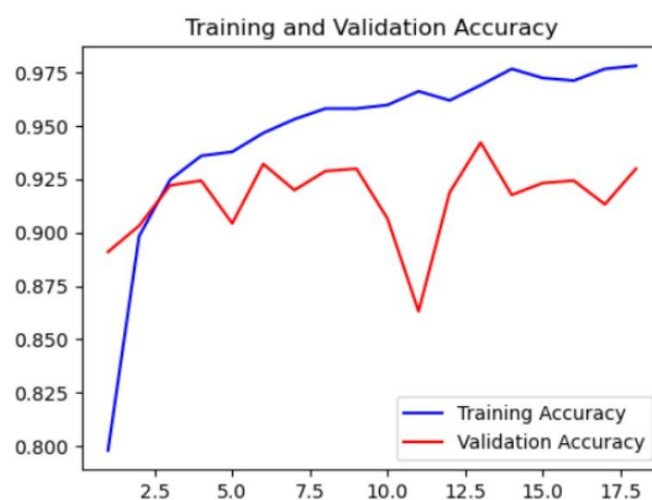


Figure 9: Accuracy vs. Epochs Curve for Deepfake Image Detection Model

- **Loss vs. Epochs:** The loss graph showed a **sharp decline in early epochs**, demonstrating that the model was successfully minimizing classification errors. As training progressed, the loss stabilized, ensuring that the model had achieved optimal performance. A small gap between training and validation loss indicated that **early stopping helped prevent overfitting**.

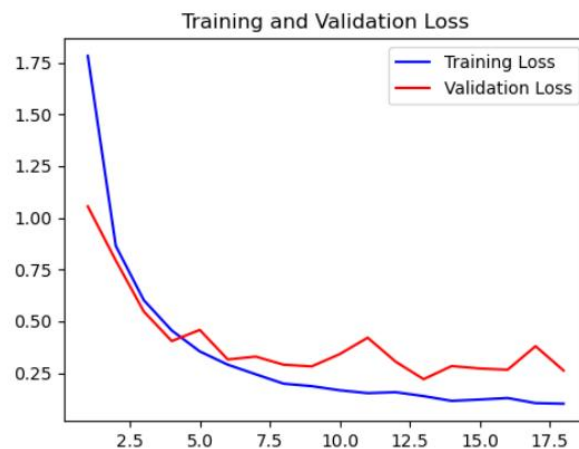


Figure 10: Loss vs. Epochs Curve for Deepfake Image Detection Model

- **Generalization Performance:** The graphs revealed that the model effectively learned patterns in the dataset without memorizing them, making it suitable for real-world application. The consistency between training and validation curves confirmed that the model was **well-tuned and reliable**.

These performance graphs provided crucial insights into the model's learning process, helping to validate its accuracy and potential for deployment.

4.5 Deployment Using Streamlit

To make the deepfake detection model accessible and user-friendly, it was deployed as a web application using Streamlit. Streamlit provided a simple and efficient way to create an interactive user interface where users could upload images and receive real-time predictions about whether an image was real or deepfake. This section details the implementation of the deployment process, including the web application interface, prediction workflow, and user interaction.

4.5.1 Web Application Interface

The Streamlit-based web application was designed to be intuitive and easy to use, allowing users to quickly upload an image and receive classification results. The key features of the interface include:

- A **clean and minimalistic UI** with a title and brief description explaining the purpose of the application.

- An **image uploader component** that allows users to select and upload images in JPG, JPEG, or PNG formats.
- A **display section** where the uploaded image is shown before processing.
- A **prediction button** that triggers the classification process, displaying the result within seconds.
- A **result output area** that shows whether the uploaded image is a real or deepfake image, along with a confidence score to indicate the model's certainty in its prediction.

The interface ensures a seamless experience for users, making deepfake detection accessible even to individuals without technical expertise.

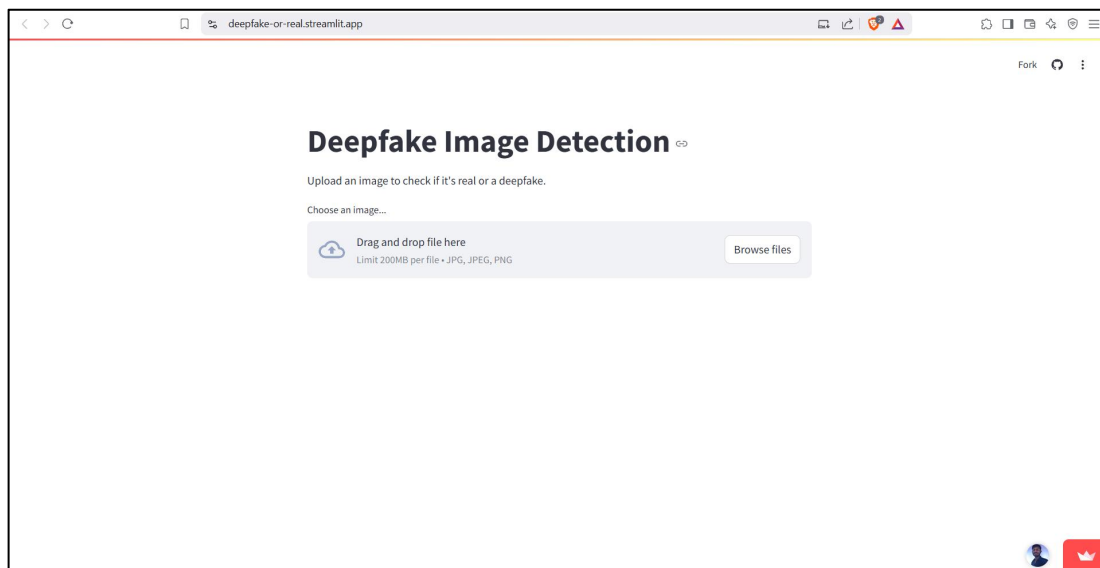


Figure11: Interface of DeepFake Image Detection

4.5.2 Prediction Workflow

Once an image is uploaded, the application follows a structured workflow to process and classify the image. The key steps include:

Image Preprocessing:

- The uploaded image is resized to **256x256 pixels** to match the input shape required by the trained model.
- Pixel values are **normalized (scaled between 0 and 1)** to improve model performance and maintain consistency.

Model Prediction:

- The preprocessed image is **passed to the trained deepfake detection model**, which makes a prediction based on learned patterns.
- The model outputs a probability score indicating whether the image is real or deepfake.

Result Interpretation:

- If the model's probability score is **above 0.5**, the image is classified as **Real**; otherwise, it is labeled as **Deepfake**.
- The **confidence percentage** is displayed to help users understand the reliability of the prediction.

This streamlined workflow ensures fast and accurate classification, making the application efficient for real-world usage.

4.5.3 User Interaction and Output Interpretation

The user interaction is designed to be straightforward, with clear instructions guiding users through the process. The following aspects enhance usability:

Uploading an Image: Users can drag and drop an image or browse their system to select an image file.

Viewing Uploaded Image: The image is displayed within the app to provide visual confirmation of the uploaded file.

Generating Prediction: A single click on the **"Predict"** button triggers the classification process.

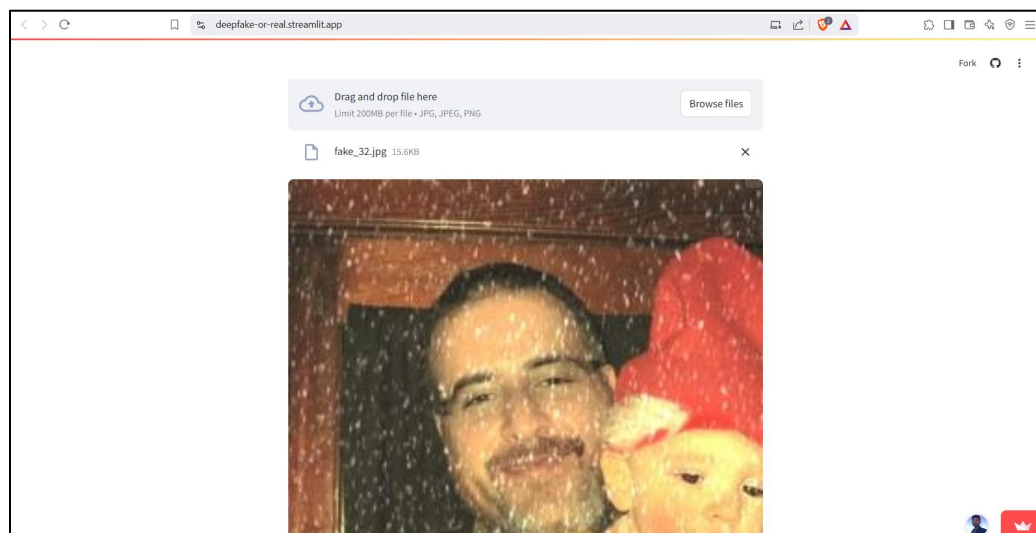


Figure 12: Deepfake detection web app interface with uploaded image

Interpreting Results:

- The app displays the **predicted label** (Real or Deepfake) prominently.
- A **confidence score** (e.g., 89.10% confidence) is shown to indicate how certain the model is about its classification.

- If needed, users can **upload another image** to perform a new prediction.

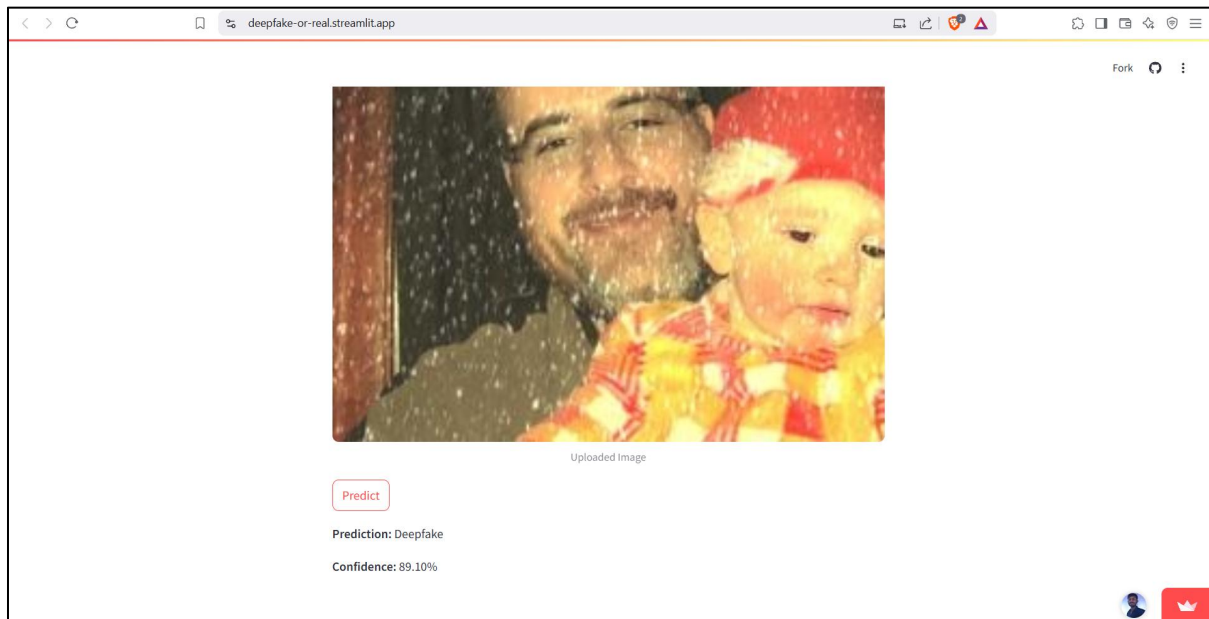


Figure 13: Deepfake Detection Web App Showing Prediction Result for the Uploaded Image

This interactive approach ensures that users can easily analyze images without requiring any technical knowledge, making deepfake detection more accessible to a wider audience.

4.6 Quality Assurance

Quality assurance is critical for the successful deployment of deepfake image detecting systems. It entails doing a thorough study of system needs to guarantee compliance with accuracy benchmarks and ethical norms. The detection algorithm's reliability and resilience are validated by rigorous code reviews, thorough model testing, and security assessments. Compliance checks verify that data protection legislation and principles of responsible AI usage are followed. Performance tweaking is required to improve detection speed and accuracy across a variety of datasets. User validation through usability testing helps gather valuable feedback, enhancing the model's interpretability and trustworthiness. Well-structured documentation and training materials provide smooth integration and adoption, while constant monitoring maintains model consistency and flexibility to developing deepfake approaches. These quality assurance measures collectively ensure that deepfake detection solutions are secure, scalable, and effective in combating digital misinformation and media manipulation.

Chapter 5

Standards Adopted

5.1 Design Standards

The deepfake image detection system was designed using a modular architecture, ensuring scalability and ease of maintenance. Convolutional Neural Networks (CNNs) were employed for their high accuracy in image classification. Standard coding conventions, directory structuring, and version control practices were followed. The design emphasized efficient preprocessing, robust model training, and clear output interpretation. Compatibility with cloud and local deployment environments was ensured. Overall, the system design prioritized performance, reliability, and user accessibility.

5.2 Coding Standards

The code was written in Python following PEP 8 guidelines for clarity and consistency. Modular design, meaningful variable names, and proper commenting were maintained. Git was used for version control, and exception handling was implemented to ensure robustness and easy debugging.

```
import os
import shutil
import tensorflow as tf
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import DenseNet121
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, GlobalAveragePooling2D
from tensorflow.keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt

# Paths and parameters
base_dir = "./data" # Correct path to your dataset
train_dir = os.path.join(base_dir, 'train')
val_dir = os.path.join(base_dir, 'val')
test_dir = os.path.join(base_dir, 'test')

IMG_HEIGHT = 256
IMG_WIDTH = 256
BATCH_SIZE = 64
EPOCHS = 21
PATIENCE = 5 # Early stopping patience
```

Figure 14: Snippet of Code

5.3 Testing Standards

The deepfake detection system was tested using standard metrics like accuracy, precision, recall, and F1-score. Unit and integration tests ensured reliable preprocessing and model performance. Cross-validation and balanced dataset splits improved result reliability. User Acceptance Testing (UAT) validated the Streamlit app's usability and real-time output.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

The deepfake detection model was successfully trained using DenseNet121, a powerful deep learning architecture renowned for its efficient feature extraction capabilities. The dataset, sourced from Kaggle, comprised 6,000 images divided into training (4,200 images), testing (900 images), and validation (900 images) sets. The model achieved an impressive 93% accuracy, showcasing its strong ability to distinguish between real and deepfake images. To address class imbalance and enhance generalization across various deepfake variations, data augmentation techniques were employed. Overfitting was effectively mitigated through strategies such as early stopping, dropout layers, and learning rate adjustments. For deployment, Streamlit was utilized to create a simple yet effective user interface, enabling users to upload images and receive real-time classification results. The evaluation metrics—including accuracy, loss, precision, recall, and F1-score—further validated the model's robust performance across diverse test cases. Overall, this project successfully demonstrated the feasibility of automated deepfake detection using deep learning-based solutions, while also highlighting areas for improvement to enhance real-world applicability.

6.2 Future Scope

To enhance robustness and scalability, future versions of the deepfake detection system can adopt advanced models like EfficientNet, Vision Transformers (ViTs), or hybrid CNN-RNN architectures. Training on larger, more diverse datasets and fine-tuning on outputs from various deepfake algorithms will improve adaptability and accuracy. Real-time deployment through web/mobile apps or REST APIs can make detection more accessible. Optimizing inference with TensorFlow Lite or ONNX will enable edge device compatibility. Expanding to video-based detection using RNNs or Transformers, and incorporating multi-modal analysis (image, speech, and text), can significantly broaden the system's capabilities. Integration with social media and forensic tools could help in curbing the spread of misinformation effectively.

References

This project was developed using a combination of research papers, publicly available datasets, and various deep learning tools and frameworks. The following references were utilized during the development of the deepfake detection model:

Datasets Used

- Kaggle Deepfake and Real Images Dataset: The dataset used for training and testing was obtained from Kaggle:
Source: [Deepfake and Real Images Dataset](#)
 - The dataset contained a mix of real and deepfake images, which were used to train and evaluate the model.

6,000 images were selected from the dataset, with a train-test-validation split of 70%-15%-15% (4,200 for training, 900 for testing, and 900 for validation)

Research Papers and Articles

1. Zobaed, S.; Rabby, F.; Hossain, I.; Hossain, E.; Hasan, S.; Karim, A.; Hasib, K.M. Deepfakes: Detecting forged and synthetic media content using machine learning. In *Artificial Intelligence in Cyber Security: Impact and Implications*; Springer: Berlin/Heidelberg, Germany, 2021; pp. 177–201. [[Google Scholar](#)]
2. Thambawita, V.; Isaksen, J.L.; Hicks, S.A.; Ghouse, J.; Ahlberg, G.; Linneberg, A.; Grarup, N.; Ellervik, C.; Olesen, M.S.; Hansen, T.; et al. DeepFake electrocardiograms using generative adversarial networks are the beginning of the end for privacy issues in medicine. *Sci. Rep.* 2021, *11*, 21869. [[Google Scholar](#)] [[CrossRef](#)]
3. Ahmed, M.F.B.; Miah, M.S.U.; Bhowmik, A.; Sulaiman, J.B. Awareness to Deepfake: A resistance mechanism to Deepfake. In Proceedings of the 2021 International Congress of Advanced Technology and Engineering (ICOTEN), Taiz, Yemen, 4–5 July 2021; pp. 1–5. [[Google Scholar](#)]
4. Gautam, N.; Vishwakarma, D.K. Obscenity Detection in Videos through a Sequential ConvNet Pipeline Classifier. *IEEE Trans. Cogn. Dev. Syst.* 2022. [[Google Scholar](#)] [[CrossRef](#)]
5. Naik, R. Deepfake Crimes: How Real and Dangerous They Are in 2021? 2021. Available online: <https://cooltechzone.com/research/deepfake-crimes> (accessed on 25 July 2022).
6. F. F. Kharbat, T. Elamsy, A. Mahmoud, and R. Abdullah, “Image feature detectors for deepfake video detection,” in Proc. IEEE/ACS 16th Int. Conf. Comput. Syst. Appl. (AICCSA), Nov. 2019, pp. 1–4.
7. Z. Akhtar and D. Dasgupta, “A comparative evaluation of local feature descriptors for DeepFakes detection,” in Proc. IEEE Int. Symp. Technol. Homeland Secur. (HST), Nov. 2019, pp. 1–5.
8. H. H. Nguyen, J. Yamagishi, and I. Echizen, “Capsule-forensics: Using capsule networks to detect forged images and videos,” in Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP), May 2019, pp. 2307–2311.
9. I. Amerini, L. Galteri, R. Caldelli, and A. D. Bimbo, “Deepfake video detection through optical flow based CNN,” in Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop (ICCVW), Oct. 2019, pp. 1205–1207.
10. M. Dordevic, M. Milivojevic, and A. Gavrovska, “DeepFake video analysis using SIFT features,” in Proc. 27th Telecommun. Forum (TELFOR), Nov. 2019, pp. 1–4.
11. Deepfake Images Detection and Reconstruction Challenge—21st International Conference on Image Analysis and Processing. Accessed: Jan. 5, 2023. [Online]. Available: <https://iplab.dmi.unict.it/Deepfake challenge/>

12. Joel Frank, Thorsten Eisenhofer, Lea Schonherr, Asja Fischer, Dorothea Kolossa, and Thorsten Holz. Leveraging frequency analysis for deep fake image recognition. 2020. 2, 4
13. Apurva Gandhi and Shomik Jain. Adversarial perturbations fool deepfake detectors. arXiv: 2003.10596, 2020. 3
14. Justin Gilmer, Ryan P Adams, Ian Goodfellow, David Andersen, and George E Dahl. Motivating the rules of the game for adversarial example research. 2018. 3, 7
15. Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014. 2
16. Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 2
17. Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, and Aleksander Madry. Adversarial examples are not bugs, they are features. In *Advances in Neural Information Processing Systems*, pages 125–136, 2019. 8
18. Darius Afchar, Vincent Nozick, Junichi Yamagishi, and Isao Echizen. 2018. Mesonet: a Compact Facial Video Forgery Detection Network. In *Proc. of WIFS*.
19. Michael Albright and Scott McCloskey. 2019. Source Generator Attribution via Inversion. In *Proc. of CVPR Workshop on Media Forensics*.
20. Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein Generative Adversarial Networks. In *Proc. of ICML*.
21. Sudipta Banerjee, Vahid Mirjalili, and Arun Ross. 2019. Spoofing PRNU Patterns of Iris Sensors While Preserving Iris Recognition. In *Proc. of ISBA*.
22. Christian F Baumgartner, Lisa M Koch, Kerem Can Tezcan, Jia Xi Ang, and Ender Konukoglu. 2018. Visual Feature Attribution Using Wasserstein GANs. In *Proc. of CVPR*.
23. Schuhmann, C., Beaumont, R., Vencu, R., Gordon, C.W., Wightman, R., Cherti, M., Coombes, T., Katta, A., Mullis, C., Wortsman, M., Schramowski, P., Kundurthy, S.R., Crowson, K., Schmidt, L., Kaczmarczyk, R., Jitsev, J.: LAION5B: An open large-scale dataset for training next generation image-text models. *NeurIPS* (2022)
24. Sha, Z., Li, Z., Yu, N., Zhang, Y.: DE-FAKE: Detection and Attribution of Fake Images Generated by Text-to-Image Diffusion Models. arXiv preprint arXiv:2210.06998 (2022)
25. Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., Ganguli, S.: Deep unsupervised learning using nonequilibrium thermodynamics. In: *ICML* (2015)
26. Tao, M., Bao, B.K., Tang, H., Xu, C.: GALIP: Generative Adversarial CLIPs for Text-to-Image Synthesis. In: *CVPR* (2023)
27. Van Den Oord, A., Kalchbrenner, N., Kavukcuoglu, K.: Pixel recurrent neural networks. In: *ICML* (2016) 40. Xiao, Y., Wei, Z., Wang, Z.: A limited memory BFGS-type method for large-scale unconstrained optimization. *Computers & Mathematics with Applications* 56(4), 1001–1009 (2008)
28. G. Antipov, M. Baccouche, and J. Dugelay, “Face aging with conditional generative adversarial networks,” in 2017 IEEE international Conference on Image Processing (ICIP). IEEE, 2017, pp. 2089–2093.
29. Z. Zhang, Y. Song, and H. Qi, “Age progression/regression by conditional adversarial autoencoder,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 5810–5818.
30. T. Karras, S. Laine, and T. Aila, “A style-based generator architecture for generative adversarial networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4401–4410.
31. T. Karras, S. Laine, M. Aittala, J. Hellsten, J. Lehtinen, and T. Aila, “Analyzing and improving the image quality of stylegan,” arXiv preprint arXiv:1912.04958, 2019.
32. S. Battiato, O. Giudice, and A. Paratore, “Multimedia forensics: discovering the history of multimedia contents,” in *Proceedings of the 17th International Conference on Computer Systems and Technologies* 2016. ACM, 2016, pp. 5–16.
33. Conference on Advanced Video and Signal Based Surveillance (AVSS), pp. 1–6 (2018). IEEE 33. E. Sabir, J. Cheng, A. Jaiswal, W. AbdAlmageed, I. Masi, P. Natarajan,

- Recurrent convolutional strategies for face manipulation detection in videos. *Interfaces (GUI)* 3(1), 80–87 (2019)
34. I. Masi, A. Killekar, R.M. Mascarenhas, S.P. Gurudatt, W. AbdAlmageed, Two-branch recurrent network for isolating deepfakes in videos. In: *European Conference on Computer Vision*, pp. 667–684 (2020). Springer
 35. H.H. Nguyen, F. Fang, J. Yamagishi, I. Echizen, Multi-task learning for detecting and segmenting manipulated facial images and videos. In: *2019 IEEE 10th International Conference on Biometrics Theory, Applications and Systems (BTAS)*, pp. 1–8 (2019). IEEE
 36. M. Du, S. Pentyala, Y. Li, X. Hu, Towards generalizable deepfake detection with locality-aware autoencoder. In: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pp. 325–334 (2020)
 37. D.M. Montserrat, H. Hao, S.K. Yarlagadda, S. Baireddy, R. Shao, J. Horvath, E. Bartusiak, J. Yang, D. Guera, F. Zhu, E.J. Delp, Deepfakes detection with automatic face weighting. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops* (2020)
 38. Y. Zheng, J. Bao, D. Chen, M. Zeng, F. Wen, Exploring temporal coherence for more general video face forgery detection. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 15044–15054 (2021)
 39. W. Zhuang, Q. Chu, Z. Tan, Q. Liu, H. Yuan, C. Miao, Z. Luo, N. Yu, UIA-VIT: unsupervised inconsistency-aware method based on vision transformer for face forgery detection. In: *European Conference on Computer Vision*, pp. 391–407 (2022). Springer
 40. H. Dang, F. Liu, J. Stehouwer, X. Liu, A.K. Jain, On the detection of digital face manipulation. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5781–5790 (2020)

These papers provided insights into deepfake generation techniques, detection methodologies, and the challenges associated with identifying manipulated content.

Tools and Frameworks Used

Several deep learning libraries and tools were used in the project for model development, training, evaluation, and deployment:

- Python: The primary programming language used for deepfake detection.
- TensorFlow & Keras:
 - Used for building and training the deep learning model (DenseNet121).
 - Provided functions for image preprocessing, augmentation, and model evaluation.
- OpenCV: Used for image processing and manipulation to enhance model training.
- Matplotlib & Seaborn: Used for visualizing model performance, including accuracy and loss curves.
- Scikit-learn:

- Provided tools for performance evaluation (precision, recall, F1-score, and confusion matrix).
 - Used for train-test-validation splitting of the dataset.
- Streamlit:
 - Used for deploying the deepfake detection model as a web application.
 - Provided an easy-to-use interface for uploading images and displaying results.
- Google Colab: Used for training the model with GPU acceleration and experimenting with different hyperparameters before deployment.

These resources played a crucial role in the successful implementation, training, and deployment of the deepfake detection system.

SAMPLE INDIVIDUAL CONTRIBUTION REPORT:

DEEPFAKE IMAGE DETECTION

ADITYA ANAND

2106005

Abstract:

This project focuses on building a Deepfake Detection System capable of distinguishing between real and AI-generated images using a Convolutional Neural Network (CNN). The objective was to create a reliable solution that could assist in identifying digital forgeries, particularly in the form of manipulated facial imagery. After training and evaluating the model, it was integrated into a web-based interface using Streamlit to allow users to test images in real time. The workflow involved phases like data preprocessing, model development, training validation, performance tuning, and final deployment. My role primarily revolved around the visualization and documentation components of the project.

Individual Contribution and Findings:

My main responsibility in this project was to design and create all the system diagrams and performance visualizations. I was in charge of developing the Class Diagram, Sequence Diagram, Activity Diagram, and the overall Flowchart—each detailing a different aspect of the system's design, architecture, and internal logic. These diagrams helped the team and evaluators understand the complete structure and flow of the application, from data input to final prediction.

In addition to system documentation, I also contributed by writing the code that visualized the model's performance during training. This included plotting training vs. validation accuracy and loss, which played a critical role in analyzing the model's behavior over time and identifying issues such as overfitting. The graphs allowed the team to make informed decisions regarding model tuning and optimization.

Through my contributions, I ensured that the project had clear, comprehensive visual documentation and interpretable performance tracking, both of which are essential for both technical clarity and future scalability.

SAMPLE INDIVIDUAL CONTRIBUTION REPORT:

DEEPFAKE IMAGE DETECTION

MD SAUD KHAN

2106039

Abstract:

With the growing threat of AI-generated deepfakes, ensuring the authenticity of digital images has become essential. This project presents a deepfake image detection system using a convolutional neural network (CNN) trained on a well-structured dataset of real and fake images. To ensure effective learning, the dataset was programmatically split into training, validation, and test sets, with data augmentation applied to improve generalization. The model achieved over 90% accuracy on unseen data. A custom testing script was developed to automate image preprocessing, model prediction, and result visualization. Together, these components enable accurate, real-time detection of manipulated images, supporting efforts against visual misinformation.

Individual Contribution and Findings:

Data Splitting and Model Testing: I contributed to two key components of the Deepfake Image Detection project: data preparation and model evaluation. For the data preparation phase, I wrote a Python script to automate the splitting of images into training (70%), validation (15%), and testing (15%) sets. This ensured balanced and randomized distribution across classes ("real" and "fake"), which is crucial for effective model training. I also integrated data augmentation techniques using ImageDataGenerator to enhance model generalization by applying transformations such as rotation, zoom, and horizontal flipping.

In the testing phase, I developed a testing script using TensorFlow and Keras to evaluate the performance of the trained CNN model. The script handles image preprocessing—resizing, normalization, and batching—followed by model prediction. I also added functionality to visualize the image alongside the predicted label ("Real" or "Deepfake") using Matplotlib, which helped in interpreting the model's decisions. These contributions played a significant role in validating the model's performance on unseen data and ensuring the reliability of predictions in real-world scenarios.

SAMPLE INDIVIDUAL CONTRIBUTION REPORT:

DEEPFAKE IMAGE DETECTION

PRIYANSHU SRIVASTAVA

2106046

Abstract:

The Deepfake Detection project aimed to develop a Convolutional Neural Network (CNN)-based model capable of accurately classifying images as either real or deepfake. The project involved a structured workflow starting from data preparation, model training, evaluation, and finally, deployment. The training phase was critical to the model's success, involving dataset organization, preprocessing, augmentation, and performance tuning. This document outlines my individual contributions to the training component, highlighting techniques used to improve model generalization and ensure robustness on unseen data.

Individual Contribution and Findings:

My primary responsibility in the Deepfake Detection project was centered around the training phase of the CNN model. I began by writing a Python script to automate the dataset preparation process, which involved organizing 6,000 images sourced from Kaggle into training (70%), validation (15%), and testing (15%) sets. This script ensured a balanced and randomized class distribution between "real" and "deepfake" images. To enhance the model's ability to generalize, I integrated data augmentation techniques using TensorFlow's ImageDataGenerator, applying random transformations such as rotation, zoom, and horizontal flipping. These augmentations helped reduce overfitting and improved model performance across different image conditions. Additionally, I contributed to hyperparameter tuning and implemented regularization strategies, including EarlyStopping, to halt training when the validation performance plateaued. My work ensured the model was well-trained, stable, and reliable, forming a strong foundation for further evaluation and deployment.

SAMPLE INDIVIDUAL CONTRIBUTION REPORT:

DEEPFAKE IMAGE DETECTION

Sudip Mondal

2106073

Abstract:

With the increasing rise of AI-generated deepfakes, verifying the authenticity of digital images has become a critical concern. This project focuses on building a deepfake image detection system using a Convolutional Neural Network (CNN). The model was trained on a structured dataset consisting of both real and manipulated images. To enhance learning and improve robustness, the dataset was carefully divided into training, validation, and testing sets, along with data augmentation methods like image flipping, rotation, and zoom. The CNN achieved over 90% accuracy on test images it had never seen before. Additionally, a custom testing workflow was developed to automate image preprocessing, prediction, and result display. Together, these elements provide a reliable solution for identifying deepfakes and contribute to the fight against digital misinformation.

Individual Contribution and Findings:

As a core team member of the Deepfake Detection project, I led the deployment pipeline by developing a fully functional web application using Streamlit. The interface was designed to be intuitive and efficient, allowing users to upload images and instantly receive predictions labeled as "Real" or "Deepfake," along with probability scores for better interpretability. On the backend, I implemented the logic for image preprocessing—resizing, normalization, and reshaping—integrated the trained CNN model, and ensured smooth model loading and real-time inference. This seamless integration between the frontend and backend made the tool practical and accessible for end-users.

In addition to deployment, I made major contributions to data management and model validation. I built a Python script to split a 6,000-image Kaggle dataset into three sets: training (70%), validation (15%), and testing (15%), maintaining class balance and randomization. To improve the model's generalizability, I used ImageDataGenerator to enrich the data with transformations like rotation, zoom, and horizontal flipping. For the evaluation step, I wrote a test script in TensorFlow and Keras that handled preprocessing and showed predictions alongside input photos in Matplotlib. These stages were critical to guarantee the model's performance, resilience, and dependability with previously unknown data.

SAMPLE INDIVIDUAL CONTRIBUTION REPORT:

DEEPFAKE IMAGE DETECTION

SURAJIT HATI

2106074

Abstract:

With the increasing rise of AI-generated deepfakes, verifying the authenticity of digital images has become a critical concern. This project focuses on building a deepfake image detection system using a Convolutional Neural Network (CNN). The model was trained on a structured dataset consisting of both real and manipulated images. To enhance learning and improve robustness, the dataset was carefully divided into training, validation, and testing sets, along with data augmentation methods like image flipping, rotation, and zoom. The CNN achieved over 90% accuracy on test images it had never seen before. Additionally, a custom testing workflow was developed to automate image preprocessing, prediction, and result display. Together, these elements provide a reliable solution for identifying deepfakes and contribute to the fight against digital misinformation.

Individual Contribution and Findings:

I played a key role in both testing and training phases of the CNN-based deepfake detection pipeline. For the testing component, I created a Python script that streamlines the process of evaluating new images. The script includes steps like resizing input images to 224x224 pixels, scaling pixel values between 0 and 1, and reshaping them to match the input shape expected by the model. After processing, the model classifies the image as either “Real” or “Deepfake.” To make interpretation easier, I also integrated Matplotlib to display each image alongside its prediction and added basic logging to record the outputs.

On the training side, I constructed the CNN using suitable layers and hyperparameters, and trained it over 21 epochs. I monitored performance by tracking training and validation accuracy as well as loss after each epoch. To make the model more generalizable, I used Keras' ImageDataGenerator to apply data augmentation techniques like rotation, zoom, and flipping. These helped the model learn from a wider variety of image styles and prevented overfitting. Small adjustments to batch size and learning rate were also made to optimize results.

SAMPLE INDIVIDUAL CONTRIBUTION REPORT:

DEEPFAKE IMAGE DETECTION

SHASHANK GUPTA

2106303

Abstract:

This project aimed to develop an efficient Deepfake Detection System using a Convolutional Neural Network (CNN) architecture to classify images as real or fake. The solution was deployed as a web application using Streamlit, allowing easy and intuitive use for end users. The project involved multiple phases, including dataset preparation, model training, evaluation, and deployment. My contribution primarily focused on the deployment of the trained model and supporting model training efforts.

Individual Contribution and Findings:

As a core team member of the Deepfake Detection project, my primary responsibility was to design and implement the deployment pipeline. I developed a fully functional web application using Streamlit, which allows users to upload an image and instantly receive a prediction on whether the image is real or a deepfake. The web interface was designed to be user-friendly and efficient, integrating all essential components such as file upload, model prediction display, and image rendering.

To ensure smooth deployment, I wrote the backend logic that handles image preprocessing (resizing, normalization, and reshaping), loads the trained CNN model, performs prediction, and displays results in an intuitive format. The output was made clear with proper classification tags ("Real" or "Deepfake") and probability scores, making the tool transparent and interpretable for users.

In addition to deployment, I also contributed to the model training phase. I helped in organizing and splitting the dataset from Kaggle, which included 6000 images divided into training, testing, and validation sets. I also participated in tuning hyperparameters, applying data augmentation, and ensuring regularization techniques like EarlyStopping were implemented to avoid overfitting.

Overall, my contribution ensured that the Deepfake Detection model was not only accurate but also accessible and practical for end-users through a smooth deployment experience.

ABSTRACT

ORIGINALITY REPORT

12%
SIMILARITY INDEX

4%
INTERNET SOURCES

10%
PUBLICATIONS

5%
STUDENT PAPERS

PRIMARY SOURCES

1

R. N. V. Jagan Mohan, B. H. V. S. Rama Krishnam Raju, V. Chandra Sekhar, T. V. K. P. Prasad. "Algorithms in Advanced Artificial Intelligence - Proceedings of International Conference on Algorithms in Advanced Artificial Intelligence (ICAAAI-2024)", CRC Press, 2025
Publication

3%

2

Mehdi Ghayoumi. "Generative Adversarial Networks in Practice", CRC Press, 2023
Publication

1%

3

"Computing Technologies for Sustainable Development", Springer Science and Business Media LLC, 2025
Publication

<1%

4

Inuwa, Muhsin Mustapha. "Real-Time Application of Deepfake for De-Identification: Privacy Preservation and Data Protection.", University of Salford (United Kingdom)
Publication

<1%

5

www.mdpi.com
Internet Source

<1%

6

www.ncbi.nlm.nih.gov
Internet Source

<1%

7

Hesham Kamal, Maggie Mashaly. "Enhanced Hybrid Deep Learning Models-Based Anomaly

<1%